MapForge -- Infrastructure Architecture Reference

This document provides detailed infrastructure architecture diagrams, specifications, and deployment guidance for the MapForge Dynamic Data Enrichment & Classification Platform. It is intended for the infrastructure team, InfoSec reviewers, and DevOps engineers responsible for provisioning, deploying, and maintaining the platform.

1. Overview

MapForge is a full-stack web application that enables analytics teams to dynamically enrich, classify, and manage data originating from external systems (primarily BigQuery). The platform replaces manual Google Sheets workflows with a structured, multi-user application that enforces data quality, provides audit trails, and automates the round-trip of data between BigQuery and the classification interface.

Core Components

Component	Technology	Role
Frontend	React 19 SPA (Vite 7, TypeScript 5.9)	Browser-based UI served as static assets from the API container
Backend API	Node.js + Express 5 (TypeScript)	REST API, authentication, business logic, WebSocket support
Background Worker	Node.js (TypeScript)	Pipeline execution, BigQuery sync, scheduled jobs, notifications
Primary Database	PostgreSQL 16	Persistent storage for exercises, classifications, audit logs, user data
Cache Layer	Redis	Session cache, query result caching, rate limiting, pub/sub for real-time updates

Component	Technology	Role
Data Warehouse	Google BigQuery	External data source and destination (client-managed, not provisioned by MapForge)
Object Storage	Cloud Storage (GCP) / S3 (AWS)	File uploads (CSV/Excel), export artifacts, backup storage

High-Level Data Flow

```
+-------------------+       +-------------------+       +-------------------+
|                   |       |                   |       |                   |
|  BigQuery         | <---> |  MapForge API     | <---> |  PostgreSQL       |
|  (Source + Dest)  |       |  + Worker         |       |  (Classifications)|
|                   |       |                   |       |                   |
+-------------------+       +---------+---------+       +-------------------+
                                      |
                            +---------+---------+
                            |                   |
                            |  Redis Cache      |
                            |                   |
                            +-------------------+
```

2. GCP Architecture (Primary -- Client Deployment)

The client operates within Google Cloud Platform. This is the target production deployment.

2.1 Architecture Diagram

```
                        +-------------------------------+
                        |        Cloud DNS              |
                        |    mapforge.<client>.com      |
                        +---------------+---------------+
                                        |
                                        | A / CNAME record
                                        |
                        +---------------v---------------+
                        |   Cloud Load Balancing        |
                        |   (External HTTPS LB)         |
                        |   - SSL termination           |
                        |   - Managed certificate       |
                        |   - URL map routing           |
                        +-------+--------------+--------+
                                |              |
                    /api/* + /*  |              | /worker (internal)
                                |              |
              +----------------v---+    +-----v------------------+
              | Cloud Run          |    | Cloud Run              |
              | "mapforge-api"     |    | "mapforge-worker"      |
              |                    |    |                        |
              | - Express 5 API    |    | - Pipeline executor    |
              | - Serves React SPA |    | - BigQuery sync jobs   |
              | - Auth endpoints   |    | - Scheduled refreshes  |
              | - WebSocket        |    | - Email notifications  |
              | - Classification   |    | - Export processing    |
              |   CRUD             |    |                        |
              +--+-----+-----+----+    +--+-----+-----+---------+
                 |     |     |            |     |     |
        +--------+  +--+--+ +------+  +--+---+ | +---+----------+
        |           |     |        |  |      | | |              |
   +----v----+ +----v---+ | +------v--v---+  | | |  +-----------v--+
   | Cloud   | |Memory- | | | Cloud       |  | | |  | BigQuery     |
   | SQL     | |store   | | | Storage     |  | | |  | (client GCP  |
   | for     | |for     | | | (GCS)       |  | | |  |  project)    |
   | Postgres| |Redis   | | |             |  | | |  |              |
   | QL 16   | |        | | | - uploads/  |  | | |  | - Source     |
   |         | |        | | | - exports/  |  | | |  |   tables     |
   | - Main  | | - Cache| | | - backups/  |  | | |  | - Destination|
   |   DB    | | - Sess | | |             |  | | |  |   tables     |
   +---------+ +--------+ | +-------------+  | | |  +--------------+
```

```
         |                  | | |
+--------v------------------v-v-v--------+
|           VPC Network                  |
|   (Private Service Connect)            |
|                                        |
|  +-----------------------------------+  |
|  | Secret Manager                    |  |
|  | - DB credentials                  |  |
|  | - Redis auth token                |  |
|  | - BigQuery service account keys   |  |
|  | - OAuth client secrets            |  |
|  | - Encryption keys (AES-256)       |  |
|  +-----------------------------------+  |
|                                        |
|  +-----------------------------------+  |
|  | Cloud Logging + Cloud Monitoring   |  |
|  | - Application logs                 |  |
|  | - Error reporting                  |  |
|  | - Uptime checks                    |  |
|  | - Custom metrics                   |  |
|  | - Alert policies                   |  |
|  +-----------------------------------+  |
+----------------------------------------+
```

2.2 Service Specifications

Cloud Run -- API Service ("mapforge-api")

Parameter	Dev	Prod
Container image	Artifact Registry: `mapforge-api:latest`	Artifact Registry: `mapforge-api:<git-sha>`
vCPUs	1	2
Memory	512 MB	1 GB
Min instances	0 (scales to zero)	1 (always-on)
Max instances	2	10
Concurrency	80	250
Request timeout	300s	300s

Parameter	Dev	Prod
Startup CPU boost	Enabled	Enabled
Ingress	All traffic (through LB)	All traffic (through LB)
VPC connector	Serverless VPC Access	Serverless VPC Access

Cloud Run -- Worker Service ("mapforge-worker")

Parameter	Dev	Prod
Container image	Artifact Registry: `mapforge-worker:latest`	Artifact Registry: `mapforge-worker:<git-sha>`
vCPUs	1	2
Memory	1 GB	2 GB
Min instances	0 (scales to zero)	0 (scales to zero)
Max instances	2	5
Concurrency	1 (job-style)	1 (job-style)
Request timeout	900s	900s
Ingress	Internal only	Internal only
VPC connector	Serverless VPC Access	Serverless VPC Access

Cloud SQL for PostgreSQL

Parameter	Dev	Prod
Instance tier	db-f1-micro (shared vCPU, 614 MB RAM)	db-g1-small (shared vCPU, 1.7 GB RAM)
PostgreSQL version	16	16
Storage	10 GB SSD	20 GB SSD (auto-increase enabled)

Parameter	Dev	Prod
High availability	Disabled	Enabled (regional)
Automated backups	Enabled (7-day retention)	Enabled (30-day retention)
Point-in-time recovery	Disabled	Enabled
Maintenance window	Any	Sunday 02:00-06:00 UTC
SSL mode	Required	Required
Private IP	Yes (via Private Service Connect)	Yes (via Private Service Connect)
Public IP	No	No

Memorystore for Redis

Parameter	Dev	Prod
Tier	Basic	Standard (with replication)
Version	7.x	7.x
Memory	1 GB	1 GB
Auth	Enabled	Enabled
Transit encryption	Enabled (TLS)	Enabled (TLS)
Network	VPC peering	VPC peering

Cloud Storage

Parameter	Dev	Prod
Bucket name	`mapforge-dev-assets`	`mapforge-prod-assets`
Location	us-central1	us-central1
Storage class	Standard	Standard
Versioning	Disabled	Enabled

Parameter	Dev	Prod
Lifecycle	Delete after 90 days	Delete after 365 days
Access control	Uniform	Uniform
Encryption	Google-managed	Customer-managed (CMEK) optional

2.3 Networking

VPC Configuration

```
VPC: mapforge-vpc
  Region: us-central1

  Subnets:
  +-------------------------------------------------------+
  | mapforge-app-subnet (10.0.1.0/24)                     |
  |   - Cloud Run VPC connector                           |
  |   - Serverless VPC Access (for Cloud Run -> VPC)      |
  |   - Private Google Access: Enabled                    |
  +-------------------------------------------------------+

  +-------------------------------------------------------+
  | mapforge-data-subnet (10.0.2.0/24)                    |
  |   - Cloud SQL (private IP)                            |
  |   - Memorystore Redis (private IP)                    |
  |   - Private Service Connect endpoints                 |
  +-------------------------------------------------------+
```

Firewall Rules

Rule Name	Direction	Source	Destination	Ports	Action
allow-cloud-run-to-sql	Ingress	VPC connector IP range	Cloud SQL IP	5432	Allow
allow-cloud-run-to-redis	Ingress	VPC connector IP range	Memorystore IP	6379	Allow
allow-health-checks	Ingress	130.211.0.0/22, 35.191.0.0/16	All	443, 8080	Allow

Rule Name	Direction	Source	Destination	Ports	Action
deny-all-ingress	Ingress	0.0.0.0/0	All	All	Deny (default)

Private Service Connections

- Cloud SQL: Accessed via Private Service Connect endpoint within the VPC. No public IP exposed.
- Memorystore: Connected via VPC peering. Accessible only from within the VPC.
- BigQuery: Accessed via the `@google-cloud/bigquery` client library using service account credentials. Traffic stays within Google's network when both MapForge and the client's BigQuery project are in GCP.

2.4 Data Flow Diagrams

Source Refresh Flow (BigQuery -> MapForge)

```
[Cloud Scheduler]
       |
       | Triggers on cron schedule (e.g., daily 02:00 UTC)
       v
[Cloud Run: Worker]
       |
       | 1. Read BigQuery source config from PostgreSQL
       | 2. Decrypt service account key (from stored_credentials)
       | 3. Execute BigQuery SQL query
       v
[BigQuery] ---(query results)---> [Worker]
       |
       | 4. Compare results against existing source_records by unique key
       | 5. Mark records: NEW, EXISTING, CHANGED, REMOVED
       | 6. Upsert source_data in PostgreSQL
       | 7. Preserve all existing classification_values
       v
[Cloud SQL: PostgreSQL]
       |
       | 8. Invalidate Redis cache for affected exercise
       v
[Memorystore: Redis]
       |
       | 9. Send notification if new records found
       v
[Notification Service (email / in-app)]
```

Classification Save Flow (User Edit -> Persist)

```
[Browser: React SPA]
       |
       | 1. User edits classification cell
       | 2. Client-side validation (debounce 300ms)
       v
[Cloud Run: API]
       |
       | PUT /api/exercises/:id/records/:recordId/classify
       | 3. Server-side validation (picklist, required, dependent rules)
       | 4. Upsert classification_values
       | 5. Insert classification_history (audit trail)
       v
[Cloud SQL: PostgreSQL]
       |
       | 6. Update Redis cache
       v
[Memorystore: Redis]
       |
       | 7. (If streaming mode) Push to BigQuery via Storage Write API
       v
[BigQuery] (optional, async)
```

Export Flow (MapForge -> BigQuery)

```
[Admin: Trigger Export]  or  [Cloud Scheduler: Cron]
       |
       v
[Cloud Run: Worker]
       |
       | 1. Load exercise config + destination config from PostgreSQL
       | 2. Query classified records (source_records JOIN classification_values)
       | 3. Apply column mapping and transforms (unpivot if matrix mode)
       v
[PostgreSQL] ---(classified data)---> [Worker]
       |
       | 4. Decrypt BigQuery credentials
       | 5. Write to destination table (MERGE / APPEND / OVERWRITE mode)
       v
[BigQuery: Destination Table]
       |
       | 6. Log export results (rows written, errors)
       v
[Cloud SQL: PostgreSQL (pipeline_runs)]
       |
       | 7. Send completion notification
       v
[Notification Service]
```

2.5 Cost Estimates (GCP)

Development Environment

Resource	Specification	Estimated Monthly Cost
Cloud Run (API)	1 vCPU, 512 MB, scales to 0	$10-20
Cloud Run (Worker)	1 vCPU, 1 GB, scales to 0	$5-10
Cloud SQL (PostgreSQL)	db-f1-micro, 10 GB SSD	$8-12
Memorystore (Redis)	Basic 1 GB	$35
Cloud Storage	< 5 GB	$0.50
Secret Manager	< 10 secrets	$0.50

Resource	Specification	Estimated Monthly Cost
Cloud Load Balancing	Forwarding rule + processing	$18-22
Cloud Logging	< 10 GB/month	$0 (free tier)
Artifact Registry	< 5 GB	$0.50
Total (Dev)		**$75-100/month**

Production Environment

Resource	Specification	Estimated Monthly Cost
Cloud Run (API)	2 vCPU, 1 GB, min 1 instance, max 10	$40-70
Cloud Run (Worker)	2 vCPU, 2 GB, scales to 0, max 5	$15-30
Cloud SQL (PostgreSQL)	db-g1-small, 20 GB SSD, HA enabled	$50-75
Memorystore (Redis)	Standard 1 GB (with replica)	$70
Cloud Storage	< 20 GB, versioning enabled	$1-2
Secret Manager	< 15 secrets	$0.50
Cloud Load Balancing	Forwarding rule + processing	$20-30
Cloud Logging	< 50 GB/month	$0-25
Cloud Monitoring	Uptime checks + alerting	$0-5
Artifact Registry	< 10 GB	$1
Total (Prod)		**$200-310/month**

Notes:

- BigQuery costs are billed to the client's GCP project, not the MapForge project. Query volume is expected to be modest ($5-20/month based on projected usage).
- Cloud Run pricing is per-request and per-vCPU-second. Dev costs are lower because the services scale to zero when idle.
- Memorystore is the largest fixed cost. If budget is constrained, Redis can run as a sidecar container in Cloud Run for dev environments (not recommended for prod).

2.6 IAM Roles and Service Accounts

Service Accounts

Service Account	Purpose	Roles
`mapforge-api@<project>.iam`	Cloud Run API service identity	Cloud SQL Client, Secret Manager Secret Accessor, Storage Object Admin, Logging Writer
`mapforge-worker@<project>.iam`	Cloud Run Worker service identity	Cloud SQL Client, Secret Manager Secret Accessor, Storage Object Admin, Logging Writer
`mapforge-deployer@<project>.iam`	CI/CD deployment (GitHub Actions)	Cloud Run Admin, Artifact Registry Writer, Service Account User, Cloud SQL Admin (for migrations)
`mapforge-monitoring@<project>.iam`	Monitoring and alerting	Monitoring Viewer, Logging Viewer

Client-Provided Service Account (BigQuery Access)

The client provides a GCP service account JSON key with BigQuery permissions. This key is encrypted (AES-256) and stored in the `stored_credentials` table. At runtime, MapForge decrypts the key and uses it to authenticate with the `@google-cloud/bigquery` client library.

Required BigQuery roles on the client's service account:

- `roles/bigquery.dataViewer` (read source tables)
- `roles/bigquery.dataEditor` (write destination tables)
- `roles/bigquery.jobUser` (run queries)

3. AWS Architecture (Alternative Deployment)

For environments where AWS is preferred or required, MapForge can be deployed on AWS with equivalent services.

3.1 Architecture Diagram

```
                        +-------------------------------+
                        |        Route 53               |
                        |    mapforge.<domain>.com      |
                        +---------------+---------------+
                                        |
                                        | A / Alias record
                                        |
                        +---------------v---------------+
                        |   Application Load Balancer   |
                        |   (ALB - HTTPS)               |
                        |   - SSL termination (ACM)     |
                        |   - Target group routing      |
                        |   - /api/* -> API service     |
                        |   - /* -> API service (SPA)   |
                        +-------+--------------+--------+
                                |              |
                Public          |              | Private
                Subnets         |              | Subnets
                                |              |
            +----------------v---+    +-----v------------------+
            | ECS Fargate        |    | ECS Fargate            |
            | "mapforge-api"     |    | "mapforge-worker"      |
            | Service            |    | Service                |
            |                    |    |                        |
            | - Express 5 API    |    | - Pipeline executor    |
            | - React SPA        |    | - BigQuery sync        |
            | - Auth endpoints   |    | - Scheduled jobs       |
            |                    |    | - Notifications        |
            | [Task: app]        |    |                        |
            | [Sidecar: redis*]  |    | [Task: worker]         |
            +--+-----+-----+----+    +--+-----+-----+---------+
               |     |     |            |     |     |
     +--------+  +--+--+ +------+  +--+---+ | +---+----------+
     |           |     |        |  |      | | |              |
+----v----+ +----v---+ | +------v--v---+  | | |  +-----------v--+
| RDS     | |Elasti- | | | S3           |  | | |  | BigQuery     |
| Postgres| |Cache   | | |              |  | | |  | (client GCP  |
| QL 16   | |Redis   | | | - uploads/   |  | | |  |  project)    |
|         | |        | | | - exports/   |  | | |  |              |
| - Multi | | - Cache| | | - backups/   |  | | |  | Accessed via |
|   AZ    | | - Sess | | |              |  | | |  | service acct |
```

```
|   (prod)| |        | | |              |  | | |  | over HTTPS   |
+---------+ +--------+ | +--------------+  | | |  +--------------+
                        |                   | | |
                +-------v-------------------v-v-v-------+
                |              VPC                      |
                |   10.0.0.0/16                         |
                |                                       |
                |  +-------------------------------------+  |
                |  | Private Subnets (10.0.3.0/24,       |  |
                |  |                  10.0.4.0/24)        |  |
                |  | - RDS instances                      |  |
                |  | - ElastiCache nodes                  |  |
                |  | - ECS tasks (Fargate)                |  |
                |  +-------------------------------------+  |
                |                                       |
                |  +-------------------------------------+  |
                |  | Public Subnets (10.0.1.0/24,        |  |
                |  |                 10.0.2.0/24)         |  |
                |  | - ALB                                |  |
                |  | - NAT Gateway                        |  |
                |  +-------------------------------------+  |
                |                                       |
                |  +-------------------------------------+  |
                |  | Secrets Manager                      |  |
                |  | - DB credentials                     |  |
                |  | - Redis auth token                   |  |
                |  | - BigQuery service account keys      |  |
                |  | - OAuth client secrets               |  |
                |  | - AES-256 encryption key             |  |
                |  +-------------------------------------+  |
                |                                       |
                |  +-------------------------------------+  |
                |  | CloudWatch                           |  |
                |  | - Application logs                   |  |
                |  | - Container metrics                  |  |
                |  | - Alarms + SNS notifications         |  |
                |  +-------------------------------------+  |
                +-----------------------------------------+

                +-----------------------------------------+
                | ECR (Elastic Container Registry)         |
                | - mapforge-api:<tag>                     |
```

```
| - mapforge-worker:<tag>                    |
+-------------------------------------------+
```

*Note: For dev environments, Redis can run as a sidecar container within the ECS task definition to reduce cost. For prod, a dedicated ElastiCache cluster is recommended.

3.2 Service Specifications

ECS Fargate -- API Service

Parameter	Dev	Prod
Container image	ECR: `mapforge-api:latest`	ECR: `mapforge-api:<git-sha>`
vCPUs	0.5	1
Memory	1 GB	2 GB
Desired count	1	2
Min/Max (auto-scaling)	1/2	2/6
Health check path	`/api/health`	`/api/health`
Deployment	Rolling update	Rolling update (min healthy 50%)
Platform version	LATEST	LATEST

ECS Fargate -- Worker Service

Parameter	Dev	Prod
Container image	ECR: `mapforge-worker:latest`	ECR: `mapforge-worker:<git-sha>`
vCPUs	0.5	1
Memory	1 GB	2 GB
Desired count	1	1
Min/Max (auto-scaling)	0/2	1/3
Deployment	Rolling update	Rolling update

Parameter	Dev	Prod
Platform version	LATEST	LATEST

RDS PostgreSQL

Parameter	Dev	Prod
Engine version	PostgreSQL 16	PostgreSQL 16
Instance class	db.t4g.micro	db.t4g.small
Storage	20 GB gp3	50 GB gp3 (auto-scaling)
Multi-AZ	No	Yes
Automated backups	7-day retention	30-day retention
Encryption at rest	Enabled (AWS-managed key)	Enabled (KMS CMK)
SSL	Required	Required
Public access	No	No
Performance Insights	Disabled	Enabled

ElastiCache Redis

Parameter	Dev	Prod
Node type	cache.t4g.micro	cache.t4g.small
Engine version	7.x	7.x
Nodes	1	2 (primary + replica)
Multi-AZ	No	Yes
Encryption at rest	Enabled	Enabled
Encryption in transit	Enabled (TLS)	Enabled (TLS)
Auth	Enabled (AUTH token)	Enabled (AUTH token)

3.3 Networking

VPC Layout

```
VPC: mapforge-vpc (10.0.0.0/16)
  Region: us-east-1

  Availability Zones: us-east-1a, us-east-1b

  Public Subnets:
  +-------------------------------------------------------+
  | mapforge-public-1a (10.0.1.0/24) -- us-east-1a        |
  |   - ALB                                                |
  |   - NAT Gateway                                        |
  +-------------------------------------------------------+
  | mapforge-public-1b (10.0.2.0/24) -- us-east-1b        |
  |   - ALB                                                |
  +-------------------------------------------------------+

  Private Subnets:
  +-------------------------------------------------------+
  | mapforge-private-1a (10.0.3.0/24) -- us-east-1a       |
  |   - ECS Fargate tasks                                  |
  |   - RDS primary                                        |
  |   - ElastiCache primary                                |
  +-------------------------------------------------------+
  | mapforge-private-1b (10.0.4.0/24) -- us-east-1b       |
  |   - ECS Fargate tasks                                  |
  |   - RDS standby (Multi-AZ)                             |
  |   - ElastiCache replica                                |
  +-------------------------------------------------------+

  NAT Gateway: 1 (dev), 2 (prod -- one per AZ)
  Internet Gateway: 1
```

Security Groups

Security Group	Inbound Rules	Outbound Rules
`sg-alb`	443 from 0.0.0.0/0 (HTTPS)	All to `sg-ecs` on 8080

Security Group	Inbound Rules	Outbound Rules
`sg-ecs`	8080 from `sg-alb`	5432 to `sg-rds`, 6379 to `sg-redis`, 443 to 0.0.0.0/0 (BigQuery/APIs)
`sg-rds`	5432 from `sg-ecs`	None
`sg-redis`	6379 from `sg-ecs`	None

3.4 Cost Estimates (AWS)

Development Environment

Resource	Specification	Estimated Monthly Cost
ECS Fargate (API)	0.5 vCPU, 1 GB, 1 task	$15-20
ECS Fargate (Worker)	0.5 vCPU, 1 GB, 1 task	$15-20
RDS PostgreSQL	db.t4g.micro, 20 GB gp3	$15-20
ElastiCache Redis	cache.t4g.micro, 1 node	$12-15
ALB	1 LB + LCU	$18-25
S3	< 5 GB	$0.50
Secrets Manager	< 10 secrets	$4
ECR	< 5 GB	$0.50
NAT Gateway	1 gateway + data	$35-45
CloudWatch	Logs + basic metrics	$0-5
Total (Dev)		**$115-155/month**

Production Environment

Resource	Specification	Estimated Monthly Cost
ECS Fargate (API)	1 vCPU, 2 GB, 2 tasks	$60-80
ECS Fargate (Worker)	1 vCPU, 2 GB, 1 task	$30-40

Resource	Specification	Estimated Monthly Cost
RDS PostgreSQL	db.t4g.small, 50 GB gp3, Multi-AZ	$50-70
ElastiCache Redis	cache.t4g.small, 2 nodes, Multi-AZ	$40-50
ALB	1 LB + LCU	$25-40
S3	< 20 GB, versioning	$1-2
Secrets Manager	< 15 secrets	$6
ECR	< 10 GB	$1
NAT Gateway	2 gateways + data	$70-90
CloudWatch	Logs + metrics + alarms	$10-20
Route 53	Hosted zone + queries	$1-2
ACM	SSL certificate	$0 (free)
Total (Prod)		**$295-400/month**

Notes:

- NAT Gateway is the single largest cost item on AWS. For dev, consider a NAT instance (t4g.nano, ~$4/month) or VPC endpoints to reduce cost.
- AWS costs are higher than GCP primarily due to NAT Gateway pricing and Fargate per-vCPU costs.

3.5 IAM Roles (AWS)

Role	Purpose	Key Policies
`mapforge-ecs-task-execution-role`	ECS task execution (pull images, write logs)	AmazonECSTaskExecutionRolePolicy, SecretsManagerReadWrite (scoped), CloudWatchLogsFullAccess
`mapforge-api-task-role`	API container runtime permissions	SecretsManagerReadWrite (scoped), S3 bucket access, RDS connect

Role	Purpose	Key Policies
`mapforge-worker-task-role`	Worker container runtime permissions	SecretsManagerReadWrite (scoped), S3 bucket access, RDS connect
`mapforge-deployer`	CI/CD deployment (GitHub Actions)	ECS deploy, ECR push, Secrets Manager read, RDS connect (migrations)

4. Shared Architecture Concerns

4.1 CI/CD Pipeline

The deployment pipeline uses GitHub Actions for both GCP and AWS targets.

```
+----------+     +-----------+     +-------------+     +------------+     +-----------+
| Git Push |---->| Run Tests |---->| Build       |---->| Push Image |---->| Deploy    |
| (main)   |     | (lint,    |     | Container   |     | to Registry|     | to Cloud  |
|          |     |  unit,    |     | (Docker)    |     | (ECR/AR)   |     | Run/ECS   |
|          |     |  e2e)     |     |             |     |            |     |           |
+----------+     +-----------+     +-------------+     +------------+     +-----------+
                                                                                |
                                                                          +-----v-----+
                                                                          | Run DB
                                                                          | Migrations
                                                                          | (Drizzle)
                                                                          +-----------+
```

Pipeline Stages

1. **Test**: Lint (ESLint), type check (tsc), unit tests (Vitest), integration tests
2. **Build**: Multi-stage Docker build (Node.js base, install deps, compile TypeScript, bundle React SPA)
3. **Push**: Tag image with git SHA, push to container registry
4. **Migrate**: Run Drizzle ORM migrations against the target database

5. **Deploy**: Update Cloud Run service revision (GCP) or ECS service task definition (AWS)
6. **Verify**: Health check against the deployed service

Branch Strategy

Branch	Target Environment	Auto-Deploy
`main`	Production	Yes (after approval gate)
`develop`	Staging	Yes
`feature/*`	Dev (preview)	Manual trigger

4.2 Database Migration Strategy

MapForge uses Drizzle ORM for schema management and migrations.

- **Migration files**: Stored in `server/src/db/migrations/` as versioned SQL files generated by `drizzle-kit`
- **Migration execution**: Run automatically during the CI/CD deploy stage, before the new container version receives traffic
- **Rollback**: Each migration has a corresponding down migration. Rollback is manual via `drizzle-kit` CLI
- **Zero-downtime migrations**: Additive schema changes (new columns, new tables) are deployed before application code that uses them. Destructive changes (drop column, rename) follow a multi-step process:
  i. Deploy code that no longer references the old schema
  ii. Run migration to remove the old schema
  iii. Verify and clean up

4.3 Monitoring and Alerting

Metrics Collected

Metric	Source	Alert Threshold
API response time (p95)	Cloud Run / ALB metrics	> 2000ms for 5 minutes
API error rate (5xx)	Cloud Run / ALB metrics	> 5% for 5 minutes
Container CPU utilization	Cloud Run / ECS metrics	> 80% for 10 minutes

Metric	Source	Alert Threshold
Container memory utilization	Cloud Run / ECS metrics	> 85% for 10 minutes
Database connections	Cloud SQL / RDS metrics	> 80% of max connections
Database CPU	Cloud SQL / RDS metrics	> 70% for 15 minutes
Database storage	Cloud SQL / RDS metrics	> 80% of allocated
Redis memory usage	Memorystore / ElastiCache	> 80% of allocated
Pipeline execution failures	Application logs	Any failure
BigQuery sync failures	Application logs	Any failure

Alert Channels

- Email to engineering team distribution list
- Slack webhook integration (if applicable)
- PagerDuty integration for production-critical alerts (optional)

4.4 Backup and Disaster Recovery

Component	Backup Method	Frequency	Retention	Recovery Point (RPO)	Recovery Time (RTO)
PostgreSQL (Dev)	Automated snapshots	Daily	7 days	24 hours	1 hour
PostgreSQL (Prod)	Automated snapshots + PITR	Continuous	30 days	< 5 minutes	30 minutes
Redis	Not backed up (cache-only data)	N/A	N/A	N/A	Minutes (cold start)
Object Storage	Versioning enabled (prod)	Continuous	365 days	N/A	Immediate
Container Images	Immutable in registry	Permanent	All tagged versions	N/A	Minutes (redeploy)

Component	Backup Method	Frequency	Retention	Recovery Point (RPO)	Recovery Time (RTO)
Secrets	Versioned in Secret Manager	On change	All versions	N/A	Immediate

Disaster Recovery Strategy

- **Database**: In production, Multi-AZ (AWS) or Regional HA (GCP) provides automatic failover. Point-in-time recovery enables restoration to any second within the retention window.
- **Application**: Container images are immutable and stored in the registry. Redeployment of any prior version takes under 5 minutes.
- **Data Loss Prevention**: Classification data lives in PostgreSQL with transaction-level durability. BigQuery exports serve as a secondary copy of enriched data.

4.5 SSL/TLS Certificates

Connection	Certificate Source	Management
Client -> Load Balancer	Google-managed SSL cert (GCP) / ACM (AWS)	Auto-renewed
Load Balancer -> Cloud Run / ECS	Internal TLS (provided by platform)	Managed by cloud provider
App -> PostgreSQL	Cloud SQL/RDS CA certificate	Enforced via `sslmode=require` in connection string
App -> Redis	In-transit encryption (TLS)	Managed by Memorystore / ElastiCache
App -> BigQuery	HTTPS (TLS 1.2+)	Managed by Google client library

4.6 Environment Strategy

Environment	Purpose	Infrastructure	Data
Local Dev	Developer workstations	Docker Compose (Postgres + Redis containers)	Seed data / fixtures

Environment	Purpose	Infrastructure	Data
Dev	Integration testing, feature previews	Minimal cloud resources (scales to zero)	Synthetic test data
Staging	Pre-production validation, UAT	Mirrors prod config at smaller scale	Anonymized copy of prod data
Production	Live workloads	Full HA configuration	Real data

Environment-specific configuration is managed through:

- Environment variables injected at container runtime
- Secrets stored in Secret Manager / Secrets Manager (per environment)
- Separate database instances per environment (no shared databases)

5. Technology Inventory

This inventory lists all technologies and their versions for InfoSec review and technology approval.

5.1 Frontend Technologies

Technology	Version	License	Purpose	Security Notes
React	19.x	MIT	UI framework	No known CVEs at time of writing
TypeScript	5.9.x	Apache-2.0	Type-safe JavaScript	Compile-time only; not shipped to browser
Vite	7.x	MIT	Build tool and dev server	Dev dependency only; not shipped to production
AG Grid Community	Latest stable	MIT	Spreadsheet/data grid UI	Client-side only; no server communication
Zustand	Latest stable	MIT	Client-side state management	Lightweight; no external dependencies

Technology	Version	License	Purpose	Security Notes
TanStack React Query	Latest stable	MIT	Server state / data fetching	Manages API request caching and deduplication
Tailwind CSS	4.x	MIT	Utility-first CSS framework	Build-time only; outputs plain CSS
React Flow	Latest stable	MIT	Pipeline DAG visualization	Client-side only
lucide-react	Latest stable	ISC	Icon library	SVG icons; no external requests

5.2 Backend Technologies

Technology	Version	License	Purpose	Security Notes
Node.js	22.x LTS	MIT	Server runtime	LTS release with active security patches
Express	5.x	MIT	HTTP framework	Mature, widely audited
TypeScript	5.9.x	Apache-2.0	Type-safe JavaScript	Compile-time only
Drizzle ORM	Latest stable	Apache-2.0	Database ORM and migrations	Parameterized queries prevent SQL injection
@google-cloud/bigquery	Latest stable	Apache-2.0	BigQuery client library	Official Google client; uses service account auth
node-cron	Latest stable	ISC	Cron scheduling	In-process job scheduling
bcrypt	Latest stable	MIT	Password hashing	Industry standard (Google OAuth is primary auth)

Technology	Version	License	Purpose	Security Notes
jsonwebtoken	Latest stable	MIT	JWT token generation/validation	Used for session tokens
helmet	Latest stable	MIT	HTTP security headers	Adds CSP, HSTS, X-Frame-Options, etc.
cors	Latest stable	MIT	CORS middleware	Configured to allow only known origins
crypto (Node.js built-in)	N/A	N/A	AES-256 encryption for stored credentials	Built-in Node.js module

5.3 Infrastructure Technologies

Technology	Version	License	Purpose	Security Notes
PostgreSQL	16.x	PostgreSQL License	Primary database	SSL required for all connections
Redis	7.x	BSD-3-Clause	Caching and session store	AUTH enabled; TLS in transit
Docker	Latest stable	Apache-2.0	Container runtime	Multi-stage builds; minimal base images
Cloud Run (GCP)	Managed	N/A	Container hosting	Auto-patched; no OS-level access
Cloud SQL (GCP)	Managed	N/A	PostgreSQL hosting	Private IP; SSL enforced; automated backups
Memorystore (GCP)	Managed	N/A	Redis hosting	VPC-internal; auth required
Artifact Registry (GCP)	Managed	N/A	Container image registry	Vulnerability scanning available
ECS Fargate (AWS)	Managed	N/A	Container hosting	No EC2 instances to manage

Technology	Version	License	Purpose	Security Notes
RDS (AWS)	Managed	N/A	PostgreSQL hosting	VPC-internal; SSL enforced; encrypted at rest
ElastiCache (AWS)	Managed	N/A	Redis hosting	VPC-internal; auth required; encrypted
ECR (AWS)	Managed	N/A	Container image registry	Image scanning available

5.4 Authentication Technologies

Technology	Version	Purpose	Security Notes
Google OAuth 2.0	N/A	Primary SSO authentication	Industry standard; no passwords stored for SSO users
Google Workspace SSO	N/A	Domain-restricted login	Only users from the client's Google Workspace domain can authenticate
JWT (JSON Web Tokens)	N/A	Session management	Short-lived access tokens (15 min) + refresh tokens (7 days)
Email/password (fallback)	N/A	Fallback for non-Google users	bcrypt hashed; passwords never stored in plaintext

5.5 Development and Build Tools

Technology	Version	Purpose	Shipped to Production
ESLint	Latest stable	Code linting	No
Prettier	Latest stable	Code formatting	No
Vitest	Latest stable	Unit/integration testing	No
Playwright	Latest stable	E2E testing	No
GitHub Actions	N/A	CI/CD pipeline	No
drizzle-kit	Latest stable	DB migration generation	No (migrations are SQL files)

6. Security Architecture

6.1 Credential Storage

All sensitive credentials are stored using a layered encryption approach:

```
+-------------------------------------------------------------+
|  Layer 1: Cloud Secret Manager                              |
|  (DB passwords, Redis tokens, OAuth secrets, master key)  |
+------------------------------+-------------------------+
                               |
                               v
+-------------------------------------------------------------+
|  Layer 2: Application-Level Encryption                      |
|  (BigQuery service account keys stored in PostgreSQL)     |
|                                                             |
|  Algorithm: AES-256-GCM                                     |
|  Key source: Master key from Secret Manager                 |
|  Storage: stored_credentials.encrypted_value (bytea)       |
|  IV: Unique per credential, stored alongside ciphertext    |
+-------------------------------------------------------------+
```

Credential Lifecycle

1. Admin uploads a BigQuery service account JSON key through the UI
2. The API server encrypts the key using AES-256-GCM with a master key retrieved from Secret Manager
3. The encrypted blob is stored in the `stored_credentials` table
4. At runtime, the worker decrypts the key, instantiates a BigQuery client, and destroys the plaintext key from memory after use
5. Credential metadata (name, type, created_by) is readable; the encrypted value is never sent to the client

6.2 Network Security

```
Internet
    |
    | HTTPS (TLS 1.2+ only)
    v
[Load Balancer] -- SSL termination, WAF rules (optional)
    |
    | Internal TLS
    v
[Cloud Run / ECS] -- Runs in private VPC
    |
    | SSL (sslmode=require)           | TLS
    v                                 v
[PostgreSQL - Private IP only]   [Redis - Private IP only]
    |
    | HTTPS (TLS 1.2+)
    v
[BigQuery API - google.com]
```

Key network security controls:

- No database or cache instances are accessible from the public internet
- All inter-service communication uses TLS
- Cloud Run / ECS tasks operate within a VPC with restricted egress
- Firewall rules / security groups follow least-privilege (only required ports open between services)
- WAF (Cloud Armor on GCP / AWS WAF) can be optionally enabled for the load balancer

6.3 Authentication Flow

```
+-------------------+         +-------------------+         +-------------------+
|                   |         |                   |         |                   |
|  Browser (SPA)    |         |  MapForge API     |         |  Google OAuth     |
|                   |         |                   |         |                   |
+--------+----------+         +--------+----------+         +--------+----------+
         |                             |                             |
         | 1. Click "Sign in with Google"                            |
         |------------------------------------------------------------>
         |                             |                             |
         | 2. Google login page        |                             |
         |<------------------------------------------------------------
         |                             |                             |
         | 3. User authenticates       |                             |
         |------------------------------------------------------------>
         |                             |                             |
         | 4. Redirect with auth code  |                             |
         |----------------------------->                             |
         |                             |                             |
         |                             | 5. Exchange code for tokens |
         |                             |------------------------------>
         |                             |                             |
         |                             | 6. Receive ID token + info  |
         |                             |<------------------------------
         |                             |                             |
         |                             | 7. Find or create user record|
         |                             |    (check domain allowlist) |
         |                             |                             |
         | 8. Return JWT (access +     |                             |
         |    refresh tokens)          |                             |
         |<-----------------------------                             |
         |                             |                             |
         | 9. Subsequent API calls     |                             |
         |    include JWT in           |                             |
         |    Authorization header     |                             |
         |----------------------------->                             |
```

Token Management

- **Access token**: JWT, 15-minute expiry, contains user ID, org ID, role
- **Refresh token**: Opaque token stored in PostgreSQL, 7-day expiry, rotated on use

- **Domain restriction**: Only Google accounts from the client's configured Workspace domain are permitted to authenticate
- **Fallback auth**: Email/password login available for non-Google users (admin-created accounts). Passwords hashed with bcrypt (cost factor 12).

6.4 Authorization Model (RBAC)

```
+-------------------+
|   Organization    |
|   (tenant scope)  |
+--------+----------+
         |
         | has_many
         v
+-------------------+     +-------------------+
|   Users           |     |   Enrichment      |
|                   |     |   Exercises        |
|  - role: admin    |     |                   |
|  - role: user     |     |                   |
+--------+----------+     +--------+----------+
         |                         |
         |    user_exercise_       |
         |    assignments          |
         +----------+---------------+
                    |
              +-----v------+
              | Assignment |
              | - role:    |
              |   editor   |
              |   viewer   |
              +------------+
```

Permission Matrix

Action	Admin	User (Editor)	User (Viewer)
Create/edit/delete exercises	Yes	No	No
Configure data sources	Yes	No	No
Manage reference tables	Yes	No	No

Action	Admin	User (Editor)	User (Viewer)
Build/run pipelines	Yes	No	No
Manage users and assignments	Yes	No	No
View assigned exercise data	Yes	Yes (assigned only)	Yes (assigned only)
Classify records	Yes	Yes (assigned only)	No
Bulk classify records	Yes	Yes (assigned only)	No
View audit log	Yes	Own changes only	No
Export data	Yes	No	No
View progress dashboard	Yes (all exercises)	Own progress only	Own progress only

6.5 Data Encryption

Data State	Encryption Method	Key Management
Data at rest (PostgreSQL)	Storage-level encryption (Cloud SQL / RDS managed)	Cloud provider managed or CMEK
Data at rest (Redis)	Encryption at rest (Memorystore / ElastiCache)	Cloud provider managed
Data at rest (Object Storage)	AES-256 server-side encryption	Cloud provider managed or CMEK
Data at rest (Credentials in DB)	AES-256-GCM application-level encryption	Master key in Secret Manager
Data in transit (client to LB)	TLS 1.2+ (HTTPS)	Managed SSL certificate
Data in transit (LB to app)	TLS (platform-managed)	Internal certificates

Data State	Encryption Method	Key Management
Data in transit (app to DB)	SSL (sslmode=require)	Cloud SQL/RDS CA certificate
Data in transit (app to Redis)	TLS	Memorystore/ElastiCache managed
Data in transit (app to BigQuery)	HTTPS (TLS 1.2+)	Google-managed

6.6 Additional Security Controls

Control	Implementation
HTTP security headers	`helmet` middleware (CSP, HSTS, X-Frame-Options, X-Content-Type-Options)
CORS	Configured to allow only the application's domain origin
Rate limiting	Redis-backed rate limiter on authentication endpoints (10 attempts / 15 min)
SQL injection prevention	Drizzle ORM parameterized queries; no raw string interpolation
XSS prevention	React built-in output escaping; no use of `dangerouslySetInnerHTML`
CSRF prevention	SameSite cookie attributes; token-based authentication (JWT in Authorization header)
Dependency scanning	GitHub Dependabot for automated vulnerability alerts on npm dependencies
Container scanning	Artifact Registry / ECR image scanning for known vulnerabilities
Audit logging	Every classification change, login event, and admin action logged with user attribution and timestamp
Secret rotation	Secrets stored in Secret Manager with versioning; rotation procedures documented in runbook

Appendix A: Container Dockerfile Reference

Both the API and Worker services share a common multi-stage build:

```
Stage 1: Build
  - Base: node:22-alpine
  - Install dependencies (npm ci)
  - Compile TypeScript (tsc)
  - Build React SPA (vite build) -- API container only
  - Prune devDependencies

Stage 2: Runtime
  - Base: node:22-alpine
  - Copy compiled JS + node_modules + static assets
  - Non-root user (node)
  - Expose port 8080
  - Health check endpoint
  - CMD: node dist/server/index.js (API) or node dist/worker/index.js (Worker)
```

Image size target: < 250 MB per container.

Appendix B: Local Development Environment

Developers run the full stack locally using Docker Compose:

```
services:
  postgres:    # PostgreSQL 16 on port 5432
  redis:       # Redis 7 on port 6379
  api:         # Node.js API with hot reload (Vite dev server proxied)
  worker:      # Node.js Worker with hot reload
```

Environment variables are loaded from a `.env.local` file (not committed to source control). A `.env.example` template is provided in the repository.

Appendix C: Service-to-Service Communication Matrix

Source	Destination	Protocol	Port	Purpose
Load Balancer	API (Cloud Run/ECS)	HTTPS	8080	Route client requests
API	PostgreSQL	SSL	5432	Read/write application data
API	Redis	TLS	6379	Cache reads/writes, session management
API	Worker	HTTPS (internal)	8080	Trigger async jobs (or via Redis pub/sub)
Worker	PostgreSQL	SSL	5432	Read/write pipeline and sync data
Worker	Redis	TLS	6379	Cache invalidation, job queue
Worker	BigQuery API	HTTPS	443	Query source data, write enriched data
Worker	Cloud Storage / S3	HTTPS	443	Store/retrieve file uploads and exports
CI/CD	Container Registry	HTTPS	443	Push container images
CI/CD	Cloud Run / ECS	HTTPS	443	Deploy new revisions
CI/CD	PostgreSQL	SSL	5432	Run database migrations